

Лабораторная работа №6

Основы информационной безопасности

Краснова Камилла Геннадьевна

Содержание

1	Цель работы	5
2	Теоретическое введение	6
3	Выполнение лабораторной работы	8
4	Выводы	18
	Список литературы	19

Список иллюстраций

3.1	проверка режима работы SELinux	8
3.2	Проверка работы Apache	9
3.3	Контекст безопасности Apache	9
3.4	Состояние переключателей SELinux	10
3.5	Статистика по политике	11
3.6	Типы поддиректорий	11
3.7	Создание файла	12
3.8	Контекст файла	12
3.9	Отображение файла	12
3.10	Изменение контекста	13
3.11	Отображение файла	14
3.12	Попытка прочесть лог-файл	14
3.13	Изменение файла	15
3.14	Изменение порта	15
3.15	Попытка прослушивания другого порта	15
3.16	Проверка лог-файлов	16
3.17	Проверка лог-файлов	16
3.18	Проверка портов	16
3.19	Перезапуск сервера	16
3.20	Проверка сервера	17
3.21	Проверка порта 81	17

Список таблиц

1 Цель работы

Развить навыки администрирования ОС Linux. Получить первое практическое знакомство с технологией SELinux¹. Проверить работу SELinx на практике совместно с веб-сервером Apache. **[course]**

2 Теоретическое введение

1. **SELinux (Security-Enhanced Linux)** обеспечивает усиление защиты путем внесения изменений как на уровне ядра, так и на уровне пространства пользователя, что превращает ее в действительно «непробиваемую» операционную систему. Впервые эта система появилась в четвертой версии CentOS, а в 5 и 6 версии реализация была существенно дополнена и улучшена.

SELinux имеет три основных режим работы:

- Enforcing: режим по умолчанию. При выборе этого режима все действия, которые каким-то образом нарушают текущую политику безопасности, будут блокироваться, а попытка нарушения будет зафиксирована в журнале.
- Permissive: в случае использования этого режима, информация о всех действиях, которые нарушают текущую политику безопасности, будут зафиксированы в журнале, но сами действия не будут заблокированы.
- Disabled: полное отключение системы принудительного контроля доступа.

Политика SELinux определяет доступ пользователей к ролям, доступ ролей к доменам и доступ доменов к типам. Контекст безопасности — все атрибуты SELinux — роли, типы и домены. Более подробно см. в [f].

2. **Apache** — это свободное программное обеспечение, с помощью которого можно создать веб-сервер. Данный продукт возник как доработанная версия другого HTTP-клиента от национального центра суперкомпьютерных приложений (NCSA).

Для чего нужен Apache сервер:

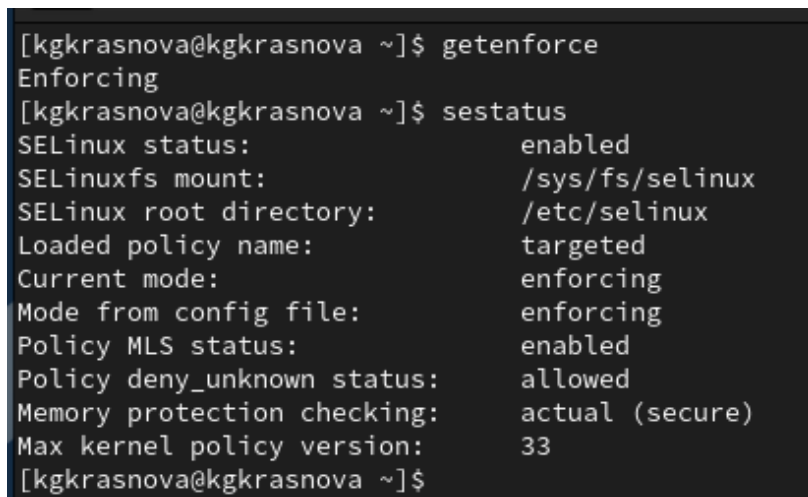
- чтобы открывать динамические PHP-страницы,
- для распределения поступающей на сервер нагрузки,
- для обеспечения отказоустойчивости сервера,
- чтобы потренироваться в настройке сервера и запуске PHP-скриптов.

Apache является кроссплатформенным ПО и поддерживает такие операционные системы, как Linux, BSD, MacOS, Microsoft, BeOS и другие.

Более подробно см. в [s].

3 Выполнение лабораторной работы

Вошла в систему под своей учетной записью. Убедилась, что SELinux работает в режиме enforcing политики targeted с помощью команд `getenforce` и `sestatus` (рис. 1).



```
[kgkrasnova@kgkrasnova ~]$ getenforce
Enforcing
[kgkrasnova@kgkrasnova ~]$ sestatus
SELinux status:                enabled
SELinuxfs mount:                /sys/fs/selinux
SELinux root directory:         /etc/selinux
Loaded policy name:              targeted
Current mode:                   enforcing
Mode from config file:          enforcing
Policy MLS status:              enabled
Policy deny_unknown status:     allowed
Memory protection checking:     actual (secure)
Max kernel policy version:      33
[kgkrasnova@kgkrasnova ~]$
```

Рисунок 3.1: проверка режима работы SELinux

Запускаю сервер `apache`, далее обращаюсь с помощью браузера к веб-серверу, запущенному на компьютере, он работает, что видно из вывода команды `service httpd status` (рис. 2).


```
[kgkrasnova@kgkrasnova ~]$ sudo systemctl start httpd
[kgkrasnova@kgkrasnova ~]$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[kgkrasnova@kgkrasnova ~]$ service httpd status
Redirecting to /bin/systemctl status httpd.service
• httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Tue 2026-04-21 09:58:49 MSK; 31s ago
     Docs: man:httpd.service(8)
   Main PID: 39835 (httpd)
   Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served: 0"
     Tasks: 177 (limit: 24606)
    Memory: 14.2M (peak: 14.8M)
       CPU: 61ms
   CGroup: /system.slice/httpd.service
           └─39835 /usr/sbin/httpd -DFOREGROUND
             └─39836 /usr/sbin/httpd -DFOREGROUND
               └─39837 /usr/sbin/httpd -DFOREGROUND
                 └─39838 /usr/sbin/httpd -DFOREGROUND
                   └─39945 /usr/sbin/httpd -DFOREGROUND

anp 21 09:58:49 kgkrasnova systemd[1]: Starting The Apache HTTP Server...
anp 21 09:58:49 kgkrasnova httpd[39835]: AH00558: httpd: could not reliably determine
anp 21 09:58:49 kgkrasnova httpd[39835]: Server configured, listening on: port 80
```

Рисунок 3.2: Проверка работы Apache

С помощью команды `ps auxZ | grep httpd` нашла веб-сервер Apache в списке процессов. Его контекст безопасности - `httpd_t` (рис. 3).

```
[kgkrasnova@kgkrasnova ~]$ ps auxZ | grep httpd
system_u:system_r:httpd_t:s0 root      39835  0.0  0.3 23984 13164 ?        Ss   09:58   0:00 /usr/sbin/httpd -DFOREGROUND
system_u:system_r:httpd_t:s0 apache  39836  0.0  0.1 23808 5420 ?        S    09:58   0:00 /usr/sbin/httpd -DFOREGROUND
system_u:system_r:httpd_t:s0 apache  39837  0.0  0.2 2162748 8864 ?        Sl   09:58   0:00 /usr/sbin/httpd -DFOREGROUND
system_u:system_r:httpd_t:s0 apache  39838  0.0  0.2 2359420 9104 ?        Sl   09:58   0:00 /usr/sbin/httpd -DFOREGROUND
system_u:system_r:httpd_t:s0 apache  39945  0.0  0.2 2228284 8868 ?        Sl   09:58   0:00 /usr/sbin/httpd -DFOREGROUND
unconfined_u:unconfined_r:unconfined_t:s0-c0.c1023 kgkrasn+ 40109 0.0  0.0 221696 2516 pts/0 S+ 10:01 0:00 grep --color=auto httpd
```

Рисунок 3.3: Контекст безопасности Apache

Просмотрела текущее состояние переключателей SELinux для Apache с помощью команды `sestatus -bigrep httpd` (рис. 4).

```

[kgkrasnova@kgkrasnova ~]$ sestatus -b httpd
SELinux status:                enabled
SELinuxfs mount:              /sys/fs/selinux
SELinux root directory:       /etc/selinux
Loaded policy name:            targeted
Current mode:                  enforcing
Mode from config file:        enforcing
Policy MLS status:             enabled
Policy deny_unknown status:    allowed
Memory protection checking:    actual (secure)
Max kernel policy version:    33

Policy booleans:
abrt_anon_write                off
abrt_handle_event              off
abrt_upload_watch_anon_write   on
antivirus_can_scan_system      off
antivirus_use_jit              off
auditadm_exec_content          on
authlogin_nsswitch_use_ldap    off
authlogin_radius               off
authlogin_yubikey              off
awstats_purge_apache_log_files off
boinc_execmem                  on
cdrecord_read_content          off
cluster_can_network_connect    off
cluster_manage_all_files       off
cluster_use_execmem            off

```

Рисунок 3.4: Состояние переключателей SELinux

Просмотрела статистику по политике с помощью команды `seinfo` (рис. 5).

```
[kgkrasnova@kgkrasnova ~]$ seinfo
Statistics for policy file: /sys/fs/selinux/policy
Policy Version:          33 (MLS enabled)
Target Policy:           selinux
Handle unknown classes:  allow

Classes:          135      Permissions:        457
Sensitivities:    1        Categories:        1024
Types:            5196     Attributes:        259
Users:            8        Roles:             15
Booleans:         361     Cond. Expr.:       393
Allow:            66166    Neverallow:        0
Auditallow:       178     Dontaudit:         8748
Type_trans:       274799  Type_change:       94
Type_member:      37      Range_trans:       5931
Role allow:       40      Role_trans:        417
Constraints:      70     Validatetrans:     0
MLS Constrain:    72     MLS Val. Tran:     0
Permissives:      4      Polcap:            6
Defaults:         7      Typebounds:        0
Allowxperm:       0      Neverallowxperm:   0
Auditallowxperm:  0      Dontauditxperm:    0
Ibendportcon:     0      Ibpkeycon:         0
Initial SIDs:     27     Fs_use:            35
Genfscon:         109    Portcon:           665
Netifcon:         0      Nodecon:           0
```

Рисунок 3.5: Статистика по политике

Типы поддиректорий, находящихся в директории /var/www, с помощью команды `ls -lZ /var/www` следующие: владелец - root, права на изменения только у владельца. Файлов в директории нет. В директории /var/www/html нет файлов.(рис. 6).

```
[kgkrasnova@kgkrasnova ~]$ ls -lZ /var/www
итого 0
drwxr-xr-x. 2 root root system_u:object_r:httpd_sys_script_exec_t:s0 6 дек 23 07:31 cgi-bin
drwxr-xr-x. 2 root root system_u:object_r:httpd_sys_content_t:s0 6 дек 23 07:31 html
[kgkrasnova@kgkrasnova ~]$ ls -lZ /var/www/html
итого 0
[kgkrasnova@kgkrasnova ~]$
```

Рисунок 3.6: Типы поддиректорий

Создать файл может только суперпользователь, поэтому от его имени создаем файл touch.html со следующим содержанием:

```
<html>
<body>test</body>
</html>
```

(рис. 7).

```
[kgkrasnova@kgkrasnova ~]$ sudo touch /var/www/html/test.html
[sudo] пароль для kgkrasnova:
[kgkrasnova@kgkrasnova ~]$ sudo nano /var/www/html/test.html
[kgkrasnova@kgkrasnova ~]$ sudo nano /var/www/html/test.html
[kgkrasnova@kgkrasnova ~]$ sudo cat /var/www/html/test.html
<html>
<body>test</body>
</html>
[kgkrasnova@kgkrasnova ~]$
```

Рисунок 3.7: Создание файла

Проверяю контекст созданного файла. По умолчанию это httpd_sys_content_t (рис. 8).

```
[kgkrasnova@kgkrasnova ~]$ ls -lZ /var/www/html/
итого 4
-rw-r--r--. 1 root root unconfined_u:object_r:httpd_sys_content_t:s0 33 anp 21 10:07 test.html
[kgkrasnova@kgkrasnova ~]$
```

Рисунок 3.8: Контекст файла

Обращаюсь к файлу через веб-сервер, введя в браузере адрес <http://127.0.0.1/test.html>. Файл был успешно отображён (рис. 9).

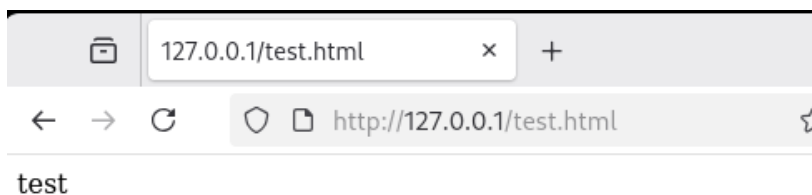


Рисунок 3.9: Отображение файла

Изучила справку `man httpd_selinux`. Рассмотрим полученный контекст детально. Так как по умолчанию пользователи CentOS являются свободными

от типа (unconfined в переводе с англ. означает свободный), созданному нами файлу test.html был сопоставлен SELinux, пользователь unconfined_u. Это первая часть контекста. Далее политика ролевого разделения доступа RBAC используется процессами, но не файлами, поэтому роли не имеют никакого значения для файлов. Роль object_r используется по умолчанию для файлов на «постоянных» носителях и на сетевых файловых системах. (В директории /proc файлы, относящиеся к процессам, могут иметь роль system_r. Если активна политика MLS, то могут использоваться и другие роли, например, secadm_r. Данный случай мы рассматривать не будем, как и предназначение :s0). Тип httpd_sys_content_t позволяет процессу httpd получить доступ к файлу. Благодаря наличию последнего типа мы получили доступ к файлу при обращении к нему через браузер.

Изменяю контекст файла /var/www/html/test.html с httpd_sys_content_t на любой другой, к которому процесс httpd не должен иметь доступа, например, на samba_share_t: `chcon -t samba_share_t /var/www/html/test.html` `ls -lZ /var/www/html/test.html` Контекст действительно поменялся (рис. [fig:010]).

```
[kgkrasnova@kgkrasnova ~]$ sudo chcon -t samba_share_t /var/www/html/test.html
[sudo] пароль для kgkrasnova:
[kgkrasnova@kgkrasnova ~]$ ls -lZ /var/www/html
итого 4
-rw-r--r--. 1 root root unconfined_u:object_r:samba_share_t:s0 33 anp 21 10:07 test.html
[kgkrasnova@kgkrasnova ~]$
```

Рисунок 3.10: Изменение контекста

При попытке отображения файла в браузере получаем сообщение об ошибке (рис. [fig:011]).

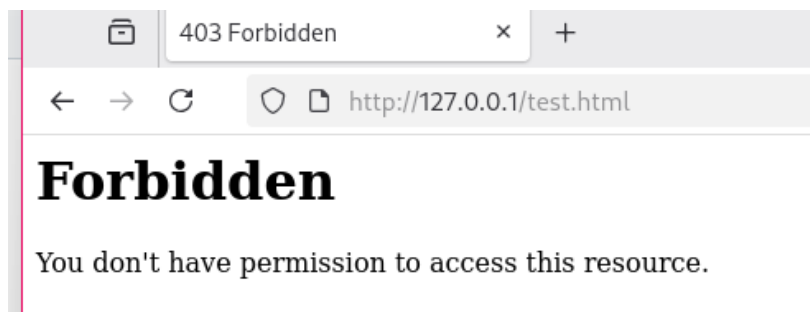


Рисунок 3.11: Отображение файла

файл не был отображён, хотя права доступа позволяют читать этот файл любому пользователю, потому что установлен контекст, к которому процесс httpd не должен иметь доступа.

Просматриваю log-файлы веб-сервера Apache и системный лог-файл: `tail /var/log/messages`. Если в системе окажутся запущенными процессы `setroubleshootd` и `auditd`, то вы также сможете увидеть ошибки, аналогичные указанным выше, в файле `/var/log/audit/audit.log`. (рис. [fig:012]).

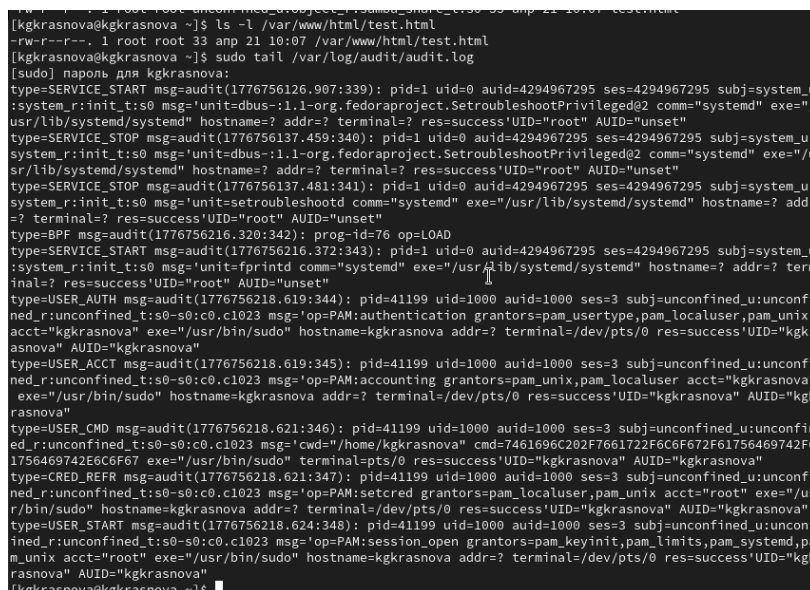


Рисунок 3.12: Попытка прочесть лог-файл

Чтобы запустить веб-сервер Apache на прослушивание TCP-порта 81 (а не 80, как рекомендует IANA и прописано в `/etc/services`) открываю файл

/etc/httpd/httpd.conf для изменения. (рис. [fig:013]).

```
kgkраснова ~$ sudo nano /etc/httpd/conf/httpd.conf
[sudo] пароль для kgkраснова:
kgkраснова@kgkраснова ~$
```

Рисунок 3.13: Изменение файла

Нахожу строчку Listen 80 и заменяю её на Listen 81. (рис. [fig:014]).

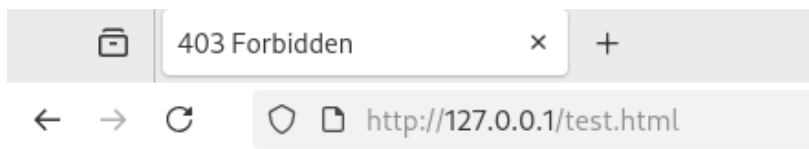
```
GNU nano 5.6.1 /etc/httpd/conf/httpd.conf
ServerRoot "/etc/httpd"

#
# Listen: Allows you to bind Apache to specific IP addresses and/or
# ports, instead of the default. See also the <VirtualHost>
# directive.
#
# Change this to Listen on a specific IP address, but note that if
# httpd.service is enabled to run at boot time, the address may not be
# available when the service starts. See the httpd.service(8) man
# page for more information.
#
#Listen 12.34.56.78:80
Listen 81

#
# Dynamic Shared Object (DSO) Support
#
# To be able to use the functionality of a module which was built as a DSO you
```

Рисунок 3.14: Изменение порта

Выполняю перезапуск веб-сервера Apache. Произошёл сбой, потому что порт 80 для локальной сети, а 81 нет (рис. [fig:015]).



Forbidden

You don't have permission to access this resource.

Рисунок 3.15: Попытка прослушивания другого порта

Проанализируйте лог-файлы: `tail -n1 /var/log/messages` (рис. [fig:016]).

```
[kgkrasnova@kgkrasnova ~]$ sudo tail -n1 /var/log/messages\
>
Apr 21 10:31:58 kgkrasnova systemd[1]: setroublehootd.service: Deactivated successfully.
[kgkrasnova@kgkrasnova ~]$
```

Рисунок 3.16: Проверка лог-файлов

Просмотрите файлы /var/log/http/error_log, /var/log/http/access_log и /var/log/audit/audit.log и выясните, в каких файлах появились записи. Запись появилась в файле error_log (рис. [fig:017]).

```
[kgkrasnova@kgkrasnova ~]$ sudo cat /var/log/httpd/error_log
[Tue Apr 21 09:58:49.866284 2026] [core:notice] [pid 39835:tid 39835] SELinux policy enabled; httpd running as context system_u:system_r:httpd_t:s0
[Tue Apr 21 09:58:49.868833 2026] [suexec:notice] [pid 39835:tid 39835] AH01232: suEXEC mechanism enabled (wrapper: /usr/sbin/suexec)
AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using fe80::a00:27f:f:fe24:d5e8%enp0s3. Set the 'ServerName' directive globally to suppress this message
[Tue Apr 21 09:58:49.891580 2026] [lbmethod:heartbeat:notice] [pid 39835:tid 39835] AH02282: No slotmem from mod_heartbeat
[Tue Apr 21 10:20:29.919632 2026] [mpm_event:notice] [pid 39835:tid 39835] AH00489: Apache/2.4.62 (Rocky Linux) configured -- resuming normal operations
[Tue Apr 21 09:58:49.895220 2026] [core:notice] [pid 39835:tid 39835] AH00094: Command line: '/usr/sbin/httpd -D FOREGROUND'
[Tue Apr 21 10:20:29.919632 2026] [core:error] [pid 39945:tid 39992] (13)Permission denied: [client 127.0.0.1:54516] AH00035: access to /test.html denied (filesystem path '/var/www/html/test.html') because search permissions are missing on a component of the path
[Tue Apr 21 10:22:06.555168 2026] [core:error] [pid 39945:tid 39993] (13)Permission denied: [client 127.0.0.1:48430] AH00035: access to /test.html denied (filesystem path '/var/www/html/test.html') because search permissions are missing on a component of the path
[Tue Apr 21 10:31:45.147733 2026] [core:error] [pid 39945:tid 39995] (13)Permission denied: [client 127.0.0.1:42116] AH00035: access to /test.html denied (filesystem path '/var/www/html/test.html') because search permissions are missing on a component of the path
[Tue Apr 21 10:31:47.393131 2026] [core:error] [pid 39945:tid 39999] (13)Permission denied: [client 127.0.0.1:42116] AH00035: access to /test.html denied (filesystem path '/var/www/html/test.html') because search permissions are missing on a component of the path
[kgkrasnova@kgkrasnova ~]$
```

Рисунок 3.17: Проверка лог-файлов

Выполняю команду `semanage port -a -t http_port_t -p tcp 81` После этого проверяю список портов командой `semanage port -l | grep http_port_t` Порт 81 появился в списке (рис. [fig:018]).

```
[kgkrasnova@kgkrasnova ~]$ sudo semanage port -a -t http_port_t -p tcp 81
Port tcp/81 already defined, modifying instead
[kgkrasnova@kgkrasnova ~]$ semanage port -l | grep http_port_t
ValueError: Политика SELinux не задана, или нет доступа к хранилищу.
[kgkrasnova@kgkrasnova ~]$ sudo semanage port -l | grep http_port_t
http_port_t      tcp      81, 80, 81, 443, 488, 8008, 8009, 8443, 9000
pegasus_http_port_t tcp      5988
[kgkrasnova@kgkrasnova ~]$
```

Рисунок 3.18: Проверка портов

Перезапускаю сервер Apache (рис. [fig:019]).

```
[kgkrasnova@kgkrasnova ~]$ sudo systemctl restart httpd
[kgkrasnova@kgkrasnova ~]$ sudo chcon -t httpd_sys_content_t /var/www/html/test.html
[kgkrasnova@kgkrasnova ~]$ sudo systemctl restart httpd
[kgkrasnova@kgkrasnova ~]$
```

Рисунок 3.19: Перезапуск сервера

Теперь он работает, ведь мы внесли порт 81 в список портов `httpd_port_t` (рис. [fig:020]).

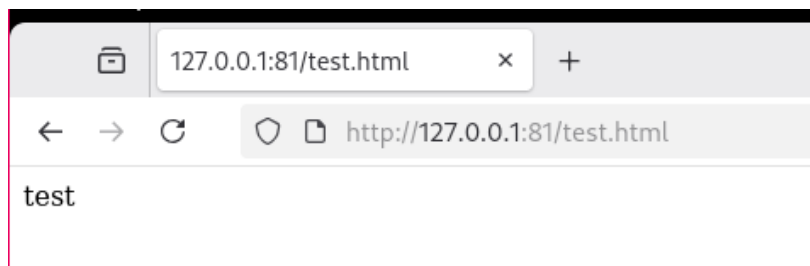


Рисунок 3.20: Проверка сервера

Возвращаю в файле `/etc/httpd/httpd.conf` порт 80, вместо 81. Проверяю, что порт 81 удален, это правда. Далее удаляю файл `test.html`, проверяю, что он удален (рис. [fig:021]).

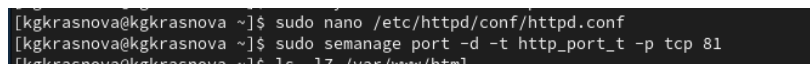


Рисунок 3.21: Проверка порта 81

4 Выводы

В ходе выполнения данной лабораторной работы были развиты навыки администрирования ОС Linux, получено первое практическое знакомство с технологией SELinux и проверена работа SELinux на практике совместно с веб-сервером Apache.

Список литературы